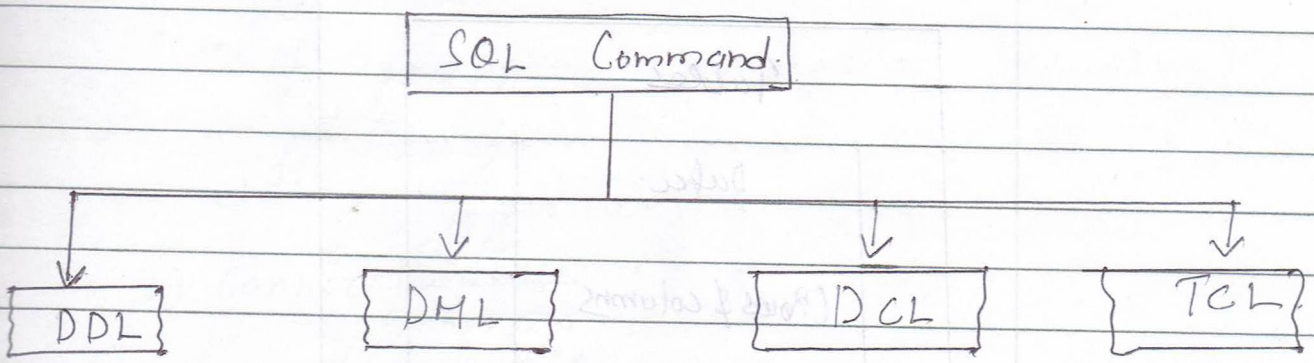
# 1) Core Database Fundamentals.

PAGE NO.....
DATE / /

## SQL Commands / SQL Commands




- DDL
  - → create
  - → Drop
  - → Alter
  - → Truncate
  - → Rename.
- DML
  - → Insert
  - → Update
  - → Delete
  - → select.
- DCL
  - → Grant
  - → Revoke
- TCL
  - → commit
  - → Roll back
  - → save point

DDL - Data Definition language.

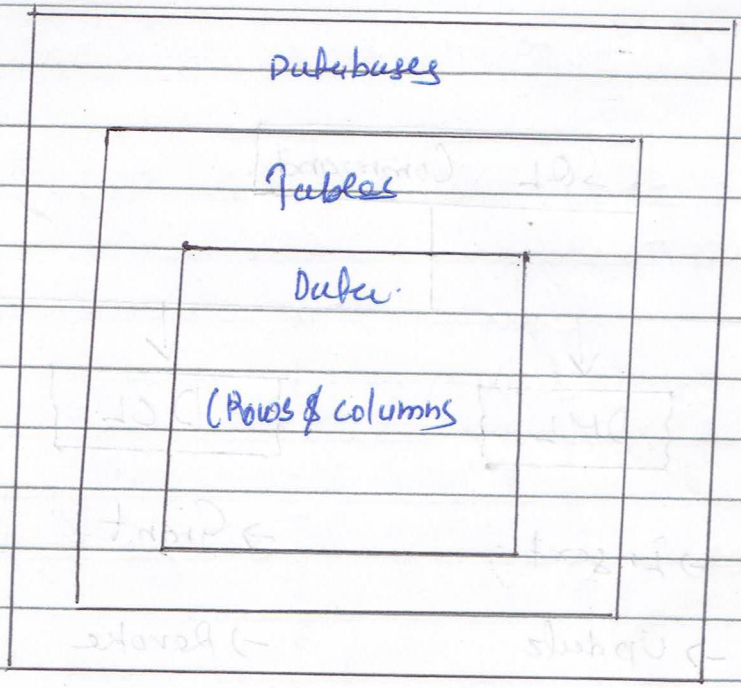
DML - Data Manipulation language.

DCL - Data Control language.

TCL - Transaction Control language.

PAGE NO.....
DATE / /

## SQL Structure / SQL structure




RDBMS

- Keys

a) Primary key

- ~~Primary key always~~
- Primary key always contains unique data.
- It cannot be null.
- There must be a single primary key.

a.(1)

| id | Name | email |
|---|---|---|
| 1 | Bhagirath | bbarg@gmail.com |
| 2 | Dawa | dawa@gmail.com |
| 3 | Sunil | sunil@gmail.com |

↳ Primary key

PAGE NO.....
DATE / /

SQL password.

bhaktaRama @ 123.

2

T1

1) Connect Database.

2) To Execute > Query.

① Yahoo Baba
② Ra is cola.

---

4.(a) primary key Syntax.

```
CREATE TABLE students (
    id    int NOT NULL AUTO_INCREMENT,
    name  VARCHAR(50) NOT NULL,
    age   INT NOT NULL,
    city  VARCHAR(10) NOT NULL,
    PRIMARY KEY (id).
```

(b) Foreign key

- The foreign key is used to link two tables.
- A foreign key in one table (child table) is used to point Primary KEY in another table (parent table).

PAGE NO.....
DATE / /

b.(1) Primary key syntax.

```
CREATE TABLE
    id   INT NOT NULL AUTO_INCREMENT,
    name VARCHAR(50), NOT NULL
    age  INT NOT NULL,
    city INT NOT NULL,
    PRIMARY KEY (id)
    FOREIGN KEY (city) REFERENCES City(id)
```

Example

Students (child table)

| id (PK) | Name | city |
|---|---|---|
| 1 | dawa | 1 |
| 2 | Sharad | 2 |
| 3 | Krishna | 1 |
| 4 | Madhu | 3 |

(FK)

Cities (parent table)

| id (PK) | Name |
|---|---|
| 1 | Sindhuli |
| 2 | Nepalgunj |
| 3 | Garas. |

(link)

8:27

PAGE NO.
DATE / /

# Tutorial 2 - Create and Drop Databases an Table - Complete MySQL.

## 1) Show databases

→ show ~~databases~~ the all database in present in system.

---

## Datatypes in MySQL

### A) String Datatypes.

1) CHAR(size) 0 to 255
2) VARCHAR(size) 0 to 65535
3) BINARY (size)
4) VARBINARY(size)
5) TINYTEXT 255 characters
6) TEXT(size) 65,535 bytes
7) MEDIUMTEXT 16,777,215 characters
8) LONGTEXT 4,294,967,295 "
9) TINYBLOB 255 bytes
10) BLOB 65,535 bytes
11) MEDIUMBLOB 16,777,215 bytes
12) LONGBLOB 4,294,967,295 "
13) ENUM (val1, val2, val3...)
14) SET (val1, val2, val3...)

### B) NUMERIC DATAtypes.

1) BIT (size) 1 to 64
2) TINYBIT(size) -128 to ~~127~~
3) INT(~~size~~(size) - 2147483648 to "
4) INTEGER(size)
5) SMALLINT(size) -32768 to "
6) MEDIUMINT(size) - 8388608 to "
7) BIGINT(size) - 19 digit to "
8) BOOL (0 or 1)
9) BOOLEAN (0 or 1)
10) FLOAT (p)
11) DOUBLE (size, d) 255.568
12) DECIMAL(size, d) size=60, d=30
13) DEC (size, d) same
14)

PAGE NO.....
DATE / /

# Date & Time Datatypes in MySQL -

1. DATE 1000-01-01 to 9999-12-31.

2) DATETIME (fsp) YYYY-MM-DD (hh:mm:ss)

3) TIMESTAMP (fsp)

4) TIME (fsp) hh:mm:ss.

5) YEAR . Four-digit format : 1901 .

PAGE NO.....
DATE / /

# 1) Basic of SQL

a) • understanding Databases.
  ↳ Diff betw^n relational and non-relation databases.
  ↳ Basic databases structure: tables, rows, columns and keys.

b) • Basic SQL Commands.
  ↳ SELECT, WHERE, ORDER BY, DISTINCT.

• understanding Databases.

| (4) | | Relational databases | | Non-Relational databases |
|---|---|---|---|---|
| | (1) | SQL type | (1) | NOSQL type. |
| structure | (2) | Table-based | (2) | • key-value pairs<br>• Document-based<br>• Graph databases<br>• wide-column stores |
| scalability | (3) | Design for scaling up vertically by upgrading one expensive custom-built hardware. | (3) | Design for scaling out horizontally by using shards to distribute load across multiple commodity (inexpensive) hardware |
| strength | (4) | Great for highly structured data and don't anticipate changes to the database structure. | (4) | pairs well with fast paced, agile development teams. |

PAGE NO.....
DATE / /

| | | | |
|---|---|---|---|
| (5) | Data stored in tables | (5) | Data stored are either key-value pairs, document-based, graph databases or wide column stores. |
| (6) | These databases have fixed or static or predefined schema. | (6) | They have dynamic schema. |
| (7) | how performance with huge volumes of data | (7) | Easily work with huge volumes of data. |
| 8 | E.g. PostgreSQL, MySQL servers. | 8 | MongoDB, Cassandra, Hbase. |

PAGE NO.....
DATE / /

## Basic SQL commands

### 1) The SELECT Statement.

→ is SQL code that retrieves data from the database, combines data from multiple tables, filter the results, perform calculation and more.

Syntax

# For selecting whole Table.

```
Select * from table_name;
```

# For selecting individual column from table.

```
Select col1, col2, col3 from table_name.
```

# For renaming individual column.

```
Select col1 as "Column1" from table_name
```

# For conditional based selecting column.

```
Select col1, col2, col3 from table_name where condition;
```

1) Where Select * from table_name where condition;

2) Select col1, col2, col3 from table_name where condition;

PAGE NO.....
DATE / /

## Where Comparison Operator

| S.N | Operator | Description |
|---|---|---|
| 1. | = | equals |
| 2. | > | greater than |
| 3 | < | less than |
| 4 | >= | Greater or equal. |
| 5 | <= | lesser or equal. |
| 6 | <> or != | Not equal. |
| 7. | BETWEEN | between a certain range. |
| 8 | LIKE | search for a pattern. |
| 9 | IN | to specify multiple possible values for a column. |
| 10 | | |

35 - 10 = 45

PAGE NO.....
DATE / /

# The Fundamental Syntax Structure of a SELECT Query.

Syntax

SELECT col1, col2 from table_name where condition
Group by col to group Having conditional filter
group by
statement that are run after grouping Order by
col to sort on.

# selecting Columns and Limiting the no. of Rows returned.

Syntax

~~Select~~

Select * From table_name Limit number

# The order by clause : Sorting Results.

ascending order

Syntax # we don't have to write ASC, without it is default

a) Select ~~col1~~ col1, col2 ~~from~~ from table_name Order by
col1 [selected column]

where [selected column] is the column that you want to sort by.

b) Select col1, col2 ... ~~from~~ from table_name Order by
[selected column] desc

# desc return results in descending order.

PAGE NO.....
DATE / /

## Introduction to simple inline calculation.

(wait for more complex)

PAGE NO.....
DATE / /

## Creating Tables.

```
Create table personal (
    id ~~INT~~ INT,
    name VARCHAR(50),
    birth-date DATE,
    phone VARCHAR(12),
    gender VARCHAR(1).
);
```

## Inserting Values in Table

```
Insert into personal (id, name, birth-date, phone, gender)

Values ( 2, "Dhawa Dorje Ghising", "1998-01-12",
        "9844272623", "M" );
```

~~To show~~

## Insert Multiple Rows Syntax.

```
Insert into table-name (col1, col2 ---)

Values
( value1, value2 ---).
(value1, value2 ---).
```

~~Implement~~

```
insert into ~~person~~ personal (id, name, birth-date, phone, gender)

values
(3, "Ram Kumar Dangang", "9
```

PAGE NO.....
DATE / /

Implament

```
insert into (id, name, phone, gender)
Values
(4, "Ram kumar Dunghang", "9800001", "M");
(5, "Salina Pokhrel", "9800002", "F");
(6, "Nischal Nawar", "9800003", "M");
```

## List of Constraints in MySQL:

- NOT NULL
- UNIQUE
- DEFAULT
- CHECK
- FOREIGN KEY
- PRIMARY KEY

## Data Table without Constraints

| id | Name | Age | Gender | Phone | City |
|---|---|---|---|---|---|
| 1 | Ram kumar | 17 | Male | 4022179 | Agra |
| 2 | Salman khan | 19 | | 4022166 | Agra |
| 3 | Meera khan | 20 | Female | 4022174 | Agra |
| | Sarita kumari | 18 | Female | 4022188 | Agra |
| 5 | Anil kapoor | 19 | Male | 4022199 | Agra |

NOT NULL  CHECK  NOT NULL  Unique  DEFAULT

PAGE NO.....
DATE / /

→ In row 2 and row 4, we have missing in "Gender" and missing in "id" respectively.

→ Some cases, while entry data there may be / can be "Age" restriction but in Column "Age" on 1st row 'Ram kumar' age is below '18' i.e. 17.

→ In column 'Phone' we have two same number at row 1 and row 3 which is dublicacy. It happens because there is no restrictions while data entry.

→ Also in Column "City", all student address are from 'Agra' which cannot accurate. There must be variety of address.

Syntax

```
create table tablename (
    id int NOT NULL,
    name varchar(50) NOT NULL,
    age int NOT NULL CHECK (age >= 18),
    gender varchar(10) NOT NULL,
    phone varchar(10) NOT NULL UNIQUE,
    city varchar(10) NOT NULL DEFAULT 'Agra',
);
```

# How to show data from Tables with SQL

PAGE NO.....
DATE / /

| Name | Age | Gender |
|---|---|---|
| Ram kumar | 19 | Male |
| Salman khan | 22 | Male |
| Meera khan | 21 | Female |
| Sarita kumari | 18 | Female |
| Anil kapoor | 22 | Male |

SELECT Syntax

```
SELECT column1, column2, column3, ..
FROM table_name;
```

```
SELECT *
FROM table_name;
```

PAGE NO.....
DATE / /

## Exercise

1) write a query that returns everything in the customers table.

2) write a query that display all of the columns and 10 rows from the customers table, sorted by customers last name, then customers first name.

3) write a query that list all customer IDs and first names in the customer table, sorted by first name.

Q.1) solution

step 1 : Create a schema Naming "customer_database"

step 2 : Create table naming "customer_info"

Sql Script

```
* Use customer_database.

* create table customer_info(
    customer_id int notnull unique,
    store_id int not null unique,
    first_name varchar(50),
    last_name varchar(50),
    email varchar(400) unique,
    address_id int not null,
    create_date date not null,
);
```

PAGE NO.....
DATE / /

# The Where Clause

# Filtering select statements Results.

Syntax

```
Select [columns to return]
from [table]

where [conditional filter statement] order by
[columns to sort on]
```

Implements

```
Select *
from customer_info

where address_id = 1 order by first_name, last_name
```

# Filtering on Multiple Conditions.

Syntax

```
Select [columns to return]
from [table]
where [condit^n] boolean Operator [condit^n]
order by [columns to sort on]
```

Implements.1) (OR)

```
select * from customer_database
where address_id = 1 or address_id = 2
order by customer_id desc;
```

PAGE NO.....
DATE / /

Implement 2 (and)

```
Select * from customer_info
where address_id = 1 and address_id = 2
order by customer_id desc;
```

(No values returned).

Implement 3 (not equal)

```
select * from customer_info
where address_id = 1 and address_id != 2
order by customer_id desc;
```

Implement 4 (not equal)

```
select * from customer_info
where address_id != 1
order by customer_id desc;
```

Implement 5 (Inbetween)

```
Select * from customer_info
where address_id > 1 and address <= 3
order by customer_id desc;
```

Implement 6 (Inbetween)

```
select * from customer_info
where customer_id between 1 and 5
order by customer_id;
```

PAGE NO.....
DATE / /

| Cond$^n$ 1 | Operator | Cond$^n$ 2 | Row Returned? |
|---|---|---|---|
| True | OR | False | True |
| True | OR | True | True |
| False | OR | False | False |
| True | AND | False | False |
| True | AND | True | True |
| True | AND NOT | False | True |
| | | | |

Implement 7 (In)

```
Select * from customer-info
where address in (1,2,3)
order by customer-id;
```

Implement 8 (Null)

PAGE NO.....
DATE / /

A subquery is a query nested within another query. Subqueries allow you to dynamically retrieve values to be used by the outer query. They can appear in the [where], [from], or [select] clauses and are commonly used for filtering, comparisons, or data transformation.